



Object Oriented Programming with Java

# Session 13 — **Exception Handling** Basics

Core Java Module

Mastering runtime errors and maintaining program stability.

Presenter:

**Shubham Rawat**



# Session Objectives

By the end of this session, you will be able to manage runtime errors and ensure application stability through Java's robust exception handling mechanisms.

## 1. Core Concept

Understand exception handling in Java and how it prevents abrupt program termination during runtime anomalies.

## 2. Hierarchy

Learn the exception hierarchy, tracing back to the Throwable class and identifying its main structural branches.

## 3. Errors vs Exceptions

Differentiate between unrecoverable system-level errors and recoverable program-level exceptions.

## 4. try-catch Blocks

Write safe and robust programs using try-catch blocks to gracefully handle and log runtime anomalies.

Course Section:

**Session 13 — Exception Handling**

# What is an Exception?

An exception is an unwanted or unexpected event that disrupts the normal flow of the program's instructions.

**Timing:** Occurs during runtime (when the program is executing).

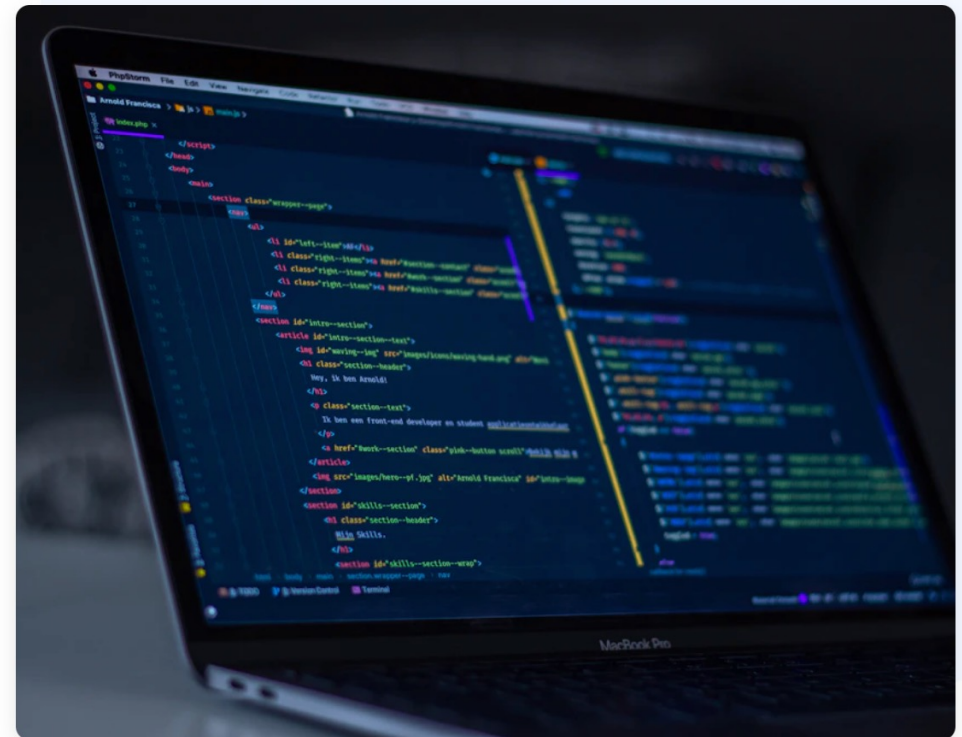
**Consequence:** Can cause immediate program termination and data loss if not properly handled.

## Common Examples:

- Divide by zero (ArithmeticException)
- Invalid array index (ArrayIndexOutOfBoundsException)
- Null reference (NullPointerException)

Course Section:

## Session 13 — Exception Handling





# Errors vs Exceptions

While both Errors and Exceptions represent anomalous conditions that disrupt normal execution, they differ significantly in their origin, severity, and how the program is expected to respond.

## ⚠ Errors (System-Level)

- **Origin:** Indicate serious system-level issues caused by the environment (JVM).
- **Handling:** Cannot usually be handled or recovered from by the application.
- **Examples:** `OutOfMemoryError`, `StackOverflowError`

## 🛡 Exceptions (Program-Level)

- **Origin:** Indicate program-level problems often caused by application logic or data.
- **Handling:** Can (and should) be caught, handled, and recovered from.
- **Examples:** `ArithmeticException`, `NullPointerException`



# Exception Hierarchy

The **Throwable** class is the root superclass of all errors and exceptions in Java. Only objects of this class or its subclasses can be thrown by the JVM.

**Error:** Represents abnormal, severe system-level conditions (like `OutOfMemoryError`) that a reasonable application should not try to catch.

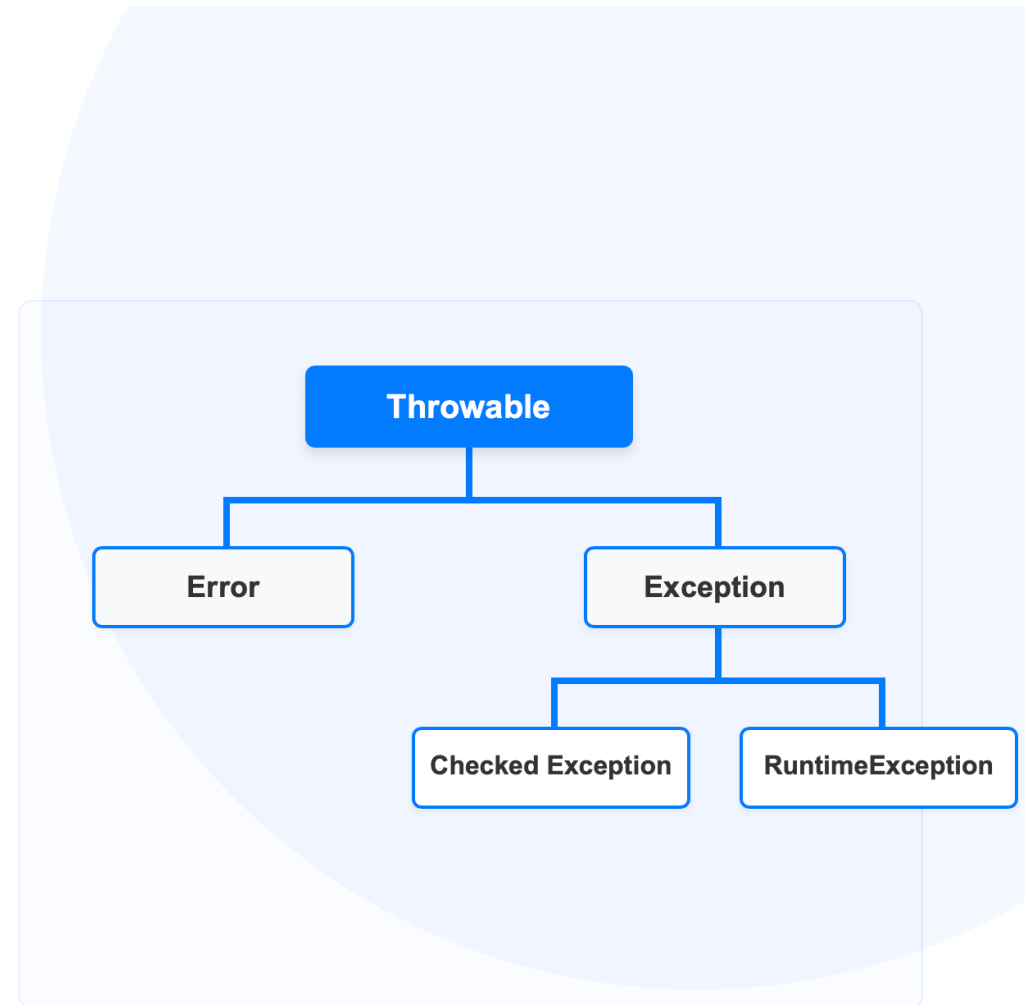
**Exception:** Represents program-level conditions that a reasonable application might want to catch and recover from.

## Exception Subdivisions:

- **Checked Exceptions:** Checked at compile-time.
- **Runtime Exceptions:** Unchecked, occur at runtime (bugs).

Course Section:

## Session 13 — Exception Handling





# Checked vs Unchecked Exceptions

Java categorizes exceptions based on when they are validated by the compiler. Understanding the difference between checked and unchecked exceptions is fundamental for designing robust and predictable application interfaces.

## ☰ Checked Exceptions (Compile-Time)

These are checked at compile time. The Java compiler forces you to either handle them using a try-catch block or declare them in the method signature using throws.

**Examples:** IOException, SQLException

## ⚡ Unchecked Exceptions (Runtime)

These occur at runtime and are subclasses of RuntimeException. They typically represent programming logic errors and do not strictly require explicit handling or declaration.

**Examples:** ArithmeticException, NullPointerException



# try-catch Block: Syntax

The **try-catch** block is the fundamental mechanism in Java for safely handling exceptions and preventing unexpected program crashes.

**try block:** Contains the "risky code" that might throw an exception. It must be followed by at least one catch or finally block.

**catch block:** Contains the error-handling logic. It "catches" the specific exception object thrown by the preceding try block.

## Key Rule:

The exception type declared in the catch parameter must match (or be a superclass of) the exception thrown.

Course Section:

## Session 13 — Exception Handling

### Java Syntax

```
try {  
    // risky code  
    // code that may throw an exception  
}  
  
catch (ExceptionType e) {  
    // handling code  
    // executes if exception occurs  
}
```



# Code Example

## Divide by Zero

When a program attempts to divide an integer by zero, the JVM automatically detects the illegal math operation and throws an [ArithmeticException](#).

By wrapping the risky operation in a `try` block, we instruct Java to monitor for errors. If the exception occurs, normal flow is interrupted, and control immediately jumps to the matching `catch` block.

### Execution Outcome:

Instead of crashing abruptly and printing a stack trace to the user, the program handles the error gracefully and continues executing the subsequent instructions safely.

Course Section:

## Session 13 — Exception Handling

DivideByZeroExample.java

```
public class DivideByZeroExample {
    public static void main(String[] args) {

        try {
            // This line will throw an exception
            int a = 10 / 0;

            // This line is skipped entirely
            System.out.println("Result is: " + a);
        }
        catch (ArithmeticException e) {
            // Exception is caught and handled here
            System.out.println("Error: Cannot divide by zero!");
        }
        System.out.println("Program executes successfully.");
    }
}
```



# Multiple Catch Blocks

Used when different exceptions may occur within the same try block, requiring distinct handling strategies.

**Evaluation:** Java evaluates catch blocks from top to bottom.

**Execution:** Only the first matching catch block executes; the remaining blocks are ignored.

## Important Rule:

Always order catch blocks from the most specific exception (subclass) to the most general exception (superclass).

```
try {
    int[] arr = new int[3];
    arr[5] = 10; // Throws Index Out of Bounds
} catch (ArithmeticException e) {
    // Handles math-related errors
    System.out.println("Math Error");
} catch (ArrayIndexOutOfBoundsException e) {
    // Handles invalid array indices
    System.out.println("Index Error");
} catch (Exception e) {
    // Catch-all for any other exceptions
    System.out.println("Generic Error");
}
```



# Hands-on Tasks

In this practical session, you will apply the core concepts of Java exception handling by writing real programs. You will learn to anticipate, catch, and gracefully handle runtime errors to build robust and fault-tolerant applications that do not crash unexpectedly.

## Task 01

### Divide-by-Zero Error

Write a Java program that performs division. Use a try-catch block to handle the `ArithmeticException` and print a user-friendly error message instead of crashing.

## Task 02

### Invalid Array Index

Create an array of fixed size and attempt to access an index beyond its bounds. Catch the resulting `ArrayIndexOutOfBoundsException` to prevent application termination.

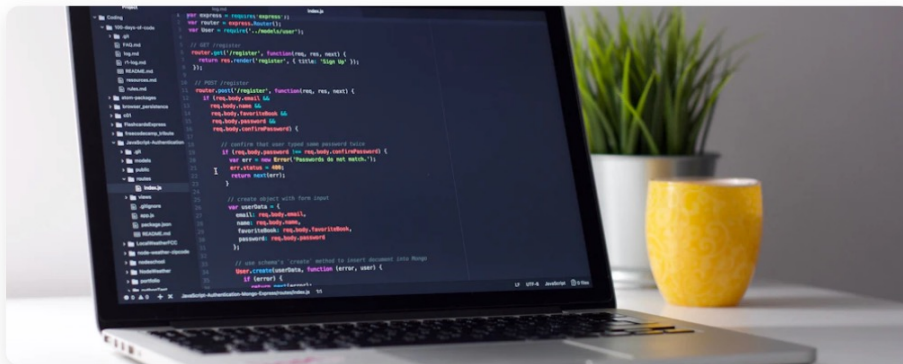
## Task 03

### Multiple Catch Blocks

Combine the previous logic into a single method. Use multiple catch blocks to independently handle both arithmetic and array index exceptions gracefully.

# Session 13 Outcome

This session concludes the fundamental concepts of Java exception handling. Students have successfully transitioned from writing vulnerable scripts to developing resilient software architectures.



Course Section:

## Session 13 — Exception Handling

### Write Safe Programs

Students are now equipped with the knowledge to write safe, fault-tolerant Java programs that maintain their execution flow even when facing unexpected anomalous scenarios.

### Proper Runtime Handling

By implementing correct try-catch structures and understanding the exception hierarchy, developers can catch and manage runtime errors before they lead to severe application crashes.

# Session 14 — **finally, throw, throws** & Custom Exceptions

Core Java Module

Advanced exception control flow and custom error signaling.

---

# finally Block — Concept

The `finally` block is an optional code block used alongside `try` and `catch` that is designed to guarantee execution of important code.

**Execution Guarantee:** It runs whether an exception occurs or not, ensuring critical final steps are never skipped.

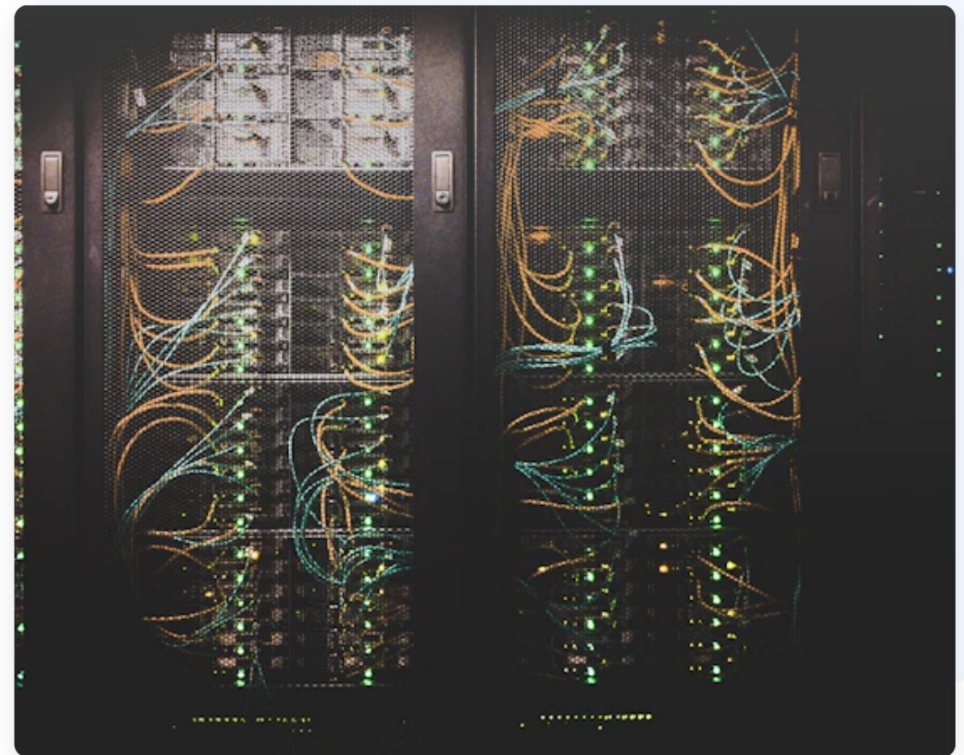
**Core Purpose:** Primarily used for mandatory cleanup operations.

## Common Cleanup Tasks:

- Closing file streams and readers
- Releasing network or socket resources
- Closing database connections

Course Section:

**Session 14 — finally, throw, throws**





# throw

## Keyword

The **throw** keyword in Java is used to manually generate and throw an explicit exception from within a method or block of code.

It is frequently utilized in validation logic to enforce business rules, immediately propagating control to the nearest matching `catch` block or caller.

### Key Characteristics:

- Followed by an instance of an exception (e.g., new Exception())
- Execution of the current method stops immediately
- Ideal for custom error conditions and input validation

Course Section:

**Session 14 — finally, throw, throws**

### Java Example

```
public void validateAge(int age) {  
    if (age < 18) {  
        // Manually throwing an exception  
        throw new ArithmeticException(  
            "Invalid operation");  
    }  
    else {  
        System.out.println("Valid age");  
    }  
}
```

# throws Keyword

The **throws** keyword is used in a method signature to declare that the method might throw exceptions of the specified type during its execution.

It explicitly shifts the responsibility of handling the exception from the current method to the **caller method**.

## Declaration Example:

```
void readFile() throws IOException {  
    // Code that may generate an IOException  
}
```

Course Section:

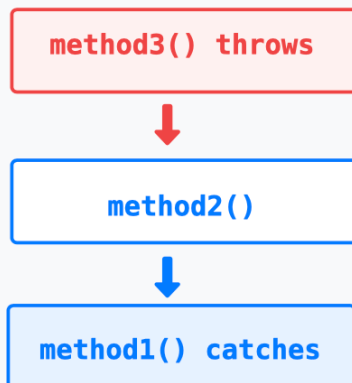
**Session 14 — finally, throw, throws**





# Exception Propagation

Exceptions pass from method to method, bubbling up the call stack until they are caught by an appropriate handler. If never caught, the program terminates.



Course Section:

**Session 14 — finally, throw, throws**

## The Call Stack

When a method throws an exception, the runtime system searches backward through the call stack to find a method that contains an appropriate exception handler (a matching catch block).

## Handling Boundaries

It is a best practice to use try-catch blocks at appropriate boundary layers of your application. This allows lower-level methods to throw exceptions, while higher-level methods manage the user experience or recovery process.



## Code — validate with throw/throws

This practical example demonstrates how to combine the method-level declaration and the manual generation of exceptions for data validation.

**throws Exception:** Placed in the method signature. It explicitly delegates the responsibility of handling the potential exception to the caller method.

**throw new Exception:** Executed inside the body when the specific business rule (age < 18) is violated, instantly disrupting the flow and generating an error object.

Course Section:

**Session 14 — finally, throw, throws**

```
void validate(int age) throws Exception {
    if (age < 18) {
        // Manually throwing an exception
        throw new Exception("Not eligible");
    } else {
        System.out.println("Eligible");
    }
}

// In caller method:
public static void main(String args[]) {
    try {
        validate(13);
    } catch (Exception e) {
        System.out.println("Caught: " + e);
    }
}
```



# Hands-on Tasks — Session 14

Apply your knowledge by completing these four practical exercises focused on manual exception generation, exception propagation, and reliable resource cleanup.

## 1. AgeValidation

Build a program that validates user age, throwing an Exception with a message "Not eligible" if the user is under 18.

## 2. throw Keyword

Demonstrate manual exception generation by throwing an `ArithmeticException` in specific invalid calculation scenarios.

## 3. throws Keyword

Implement file operations to demonstrate how an `IOException` is propagated to the caller method using 'throws'.

## 4. finally Block

Practice writing code that ensures resource cleanup (like closing a database connection) regardless of exceptions.

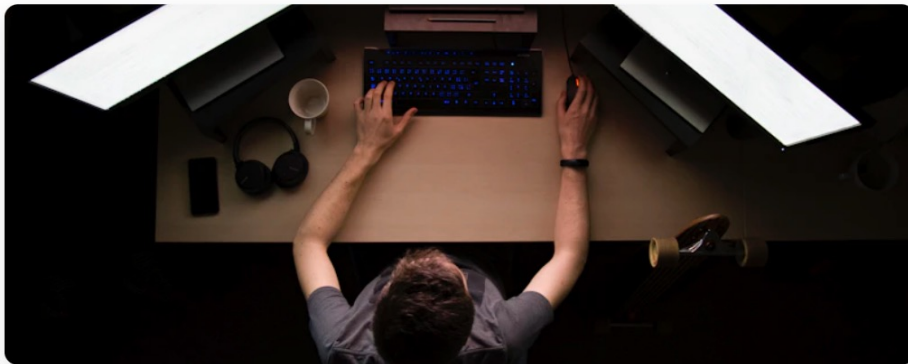
Course Section:

**Session 14 — finally, throw, throws**



# Session 14 Outcome

Students now possess a deep understanding of advanced exception control flow, enabling them to construct robust applications that gracefully handle unexpected states and strictly enforce business logic rules.



Course Section:

**Session 14 — finally, throw, throws**



## Exception Propagation

Mastery of how exceptions bubble up the call stack, ensuring errors are systematically caught and handled at the appropriate architectural boundaries without silent failures.



## Manual Exception Generation

Competence in utilizing the **throw** and **throws** keywords to proactively validate application states, reject invalid input, and correctly signal failure conditions to calling methods.



# Session 15 — Custom Exceptions (Part 2)

Core Java Module

Designing domain-specific exceptions to improve clarity and error handling.

---



# What are Custom Exceptions?

Custom exceptions are user-defined exception classes created by programmers to handle specific business logic or domain rules.

**Purpose:** Used for domain-specific validation when standard Java exceptions (like `IllegalArgumentException`) are not descriptive enough.

**Benefit:** They greatly improve code readability, simplify debugging, and allow for highly targeted error handling strategies.

## Domain-Specific Examples:

- **LoginException:** Thrown for invalid credentials.
- **PasswordException:** Thrown when strength rules aren't met.
- **InsufficientFundsException:** Thrown in banking apps.

Course Section:

**Session 15 — Custom Exceptions**





# Creating a Custom Exception

Creating a custom exception allows you to define specific error conditions relevant to your application's domain. This improves code readability and makes debugging easier by providing precise error contexts.

## Key Steps:

1. Create a new Java class.
2. **Extend `Exception`** (for checked) or **`RuntimeException`** (for unchecked).
3. Provide a constructor to pass the error message to the superclass via `super(msg)`.

LoginException.java

```
class LoginException extends Exception {  
  
    // Constructor accepting a message  
    LoginException(String msg) {  
        // Pass message to parent  
        Exception class  
        super(msg);  
    }  
  
}
```



# Using Custom Exceptions

To implement a custom exception, apply the same `throw` and `throws` keywords used with standard Java exceptions.

**Validation Logic:** Use conditional statements to check for invalid business states, then instantiate and throw the exception.

**Method Signature:** If your custom exception is checked, the method must declare it using the `throws` keyword.

## Key Takeaways:

- Throw manually when domain or business rules are violated
- Delegate the error handling to the caller via `throws`
- Keep provided error messages clear and context-specific

Course Section:

## Session 15 — Custom Exceptions

```
// Using the custom exception in a method
public void login(String user) throws
LoginException {
    // Check domain-specific rule
    if (!"admin".equals(user)) {
        // Throw the custom exception with a message
        throw new LoginException("User invalid");
    }
    System.out.println("Login successful!");
}
```



# Benefits of Custom Exceptions

Creating custom exceptions allows developers to represent domain-specific problems clearly. While standard Java exceptions cover generic errors, user-defined exceptions bridge the gap between technical failures and business logic, leading to more robust and maintainable applications.

## Error Messages

Provides clear, contextual, and descriptive error messages that relate directly to the application's unique domain, rather than relying on generic technical terminology.

## Domain Validation

Enforces critical business rules accurately by throwing exceptions that mirror business constraints (e.g., `InsufficientFundsException`) directly in the code.

## Cleaner Code

Separates explicit error-handling paths from regular execution flows, which ultimately makes the main business logic cleaner, easier to read, and simpler to test.

## Easier Debugging

Meaningful exception class names make it immediately clear what went wrong when reviewing server logs and reading stack traces during production incidents.



# Hands-on Tasks — Session 15

Apply your knowledge of custom exceptions by completing the following practical exercises to build robust, domain-specific validation logic.

## 1. LoginException

Create a custom exception for login validation. Write a program that throws this exception with a clear message when a user provides incorrect credentials or a non-existent username during authentication.

## 2. PasswordStrengthException

Implement a checked exception to enforce password policies. Trigger it when a user attempts to set a new password that lacks required complexity (e.g., missing numbers or special characters).

## 3. InvalidProductException

Design an unchecked exception for an e-commerce context. Throw it when a method attempts to add a product to a cart with invalid attributes, such as a negative price or a missing product ID.

Course Section:

**Session 15 — Custom Exceptions**



# Best Practices — Custom Exceptions

Designing domain-specific exceptions can greatly improve code clarity and error handling capabilities. However, it is important to follow established guidelines to maintain clean, readable, and manageable codebases.

## ✓ Guidelines (Do's)

- **Use meaningful exception names** (e.g., `InvalidAgeException`) to convey intent clearly.
- **Extend appropriately:** Inherit from `Exception` for checked or `RuntimeException` for unchecked exceptions.
- **Provide clear messages:** Supply actionable error messages to aid in debugging.

Course Section:

**Session 15 — Custom Exceptions**

## ⚠ Anti-Patterns (Avoid)

- **Excessive custom exceptions:** Don't create new classes if standard Java exceptions (like `IllegalArgumentException`) suffice.
- **Swallowing exceptions:** Never leave a catch block completely empty without logging or handling.
- **Overusing checked exceptions:** Avoid forcing callers to handle exceptions that represent unrecoverable programming errors.